

Helping Hand

An Assistive Robotic Platform with Vision and Voice Control

Capstone Final Report – Team B2

Team Members:

Mark Yang
Huaqing Wang
Riley Intihar
Krish Gupta
Nicole Contreras
Henry Gao

Advisor

Bahram Shafai

Table of Contents

Abstract....3

Introduction....4

Problem Formulation....5

Analysis of Major Systems....6

- I/O Analysis....6
- Software Integration....7
- Hardware Architecture....8
- Robot Operating System 2 (ROS2)....8

Design Strategies and Approach....10

- Software Components....10
- Hardware Components....12

Parts and Implementation....16

- Eye Tracking System....16
- Voice Control System....18
- RoArm-M3-Pro Vision-Guided Manipulation System....19
- Finite State Machine (FSM)....21
- Hardware....22
- Battery and Power System....24

Testing Procedures and Design Validation....30

- Eye Tracking Testing....30
- Robot Arm Testing....31
- Full System Integration Testing....33

Cost Analysis....35

Conclusion....36

Compliance....37

References....38

I Abstract

Millions of individuals worldwide suffer from profound motor impairments, with quadriplegia alone accounting for nearly 60% of all spinal cord injuries. For this population, the total loss of upper limb mobility creates a devastating barrier to independence, rendering even the most basic activities of daily living impossible without constant assistance. To address this critical loss of autonomy, this project introduces a vision-controlled robotic platform designed to restore agency through autonomous object retrieval. By integrating a motorized robotic arm with a custom eye-tracking interface and voice command module, the system translates a user's gaze or spoken input into precise control signals, bridging the gap between human intent and physical action.

To achieve the design goal, three components were designed. A glass frame was 3D printed and fitted with two CSI cameras, one facing outward toward the environment and one directed toward the user. The back camera tracks visible AprilTags in real time. The front-facing camera performs pupil-based gaze estimation with predefined regions using image processing. The software utilizes polynomial fitting based on recorded data from the calibration process to map the pupil locations to target gaze coordinates. Once the software detects 3 blinking motions in 5 seconds, it will send a ROS message to communicate with the robotic arm.

The project uses open-source ROS2 packages to control a Roarm-M3 Pro robotic arm. An end-effector-mounted camera localizes the target AprilTag, using TF trees to translate the pose into the world coordinate frame. Visual servoing then guides the arm to grasp and deliver the object.

As the system is intended to be portable and mounted to a wheelchair, the robotic arm and Jetson Orin Nano require a portable power source. To address this, a custom three-stage power conversion and delivery board was developed. This board charges a 12V 9Ah SLA battery and provides up to 12V, 6A of output. It accepts 120V AC mains, which passes through an EMI stage and rectification to 160-170V DC, before being stepped down to 14.4V via a custom DCM flyback converter. A buck-boost stage ensures a stable 12V output regardless of the battery's charge state, supplying both the arm and the computer. The system supports two input methods — eye-tracking with blink detection and voice commands — making it accessible to a broader range of users.

Testing showed the system is functional, successfully detecting pupils, responding to blinks, and retrieving book-shaped objects. However, it lacked precision and consistency, with pupil detection occasionally drifting. This limitation likely stems from the hardware; while conventional cameras were used, industry-standard infrared sensors would offer better precision across various lighting conditions. Additionally, limited calibration data occasionally caused gaze fluctuations despite the user focusing on a target.

II Introduction

According to a report published by the World Health Organization, about 15% of the worldwide population live with some form of disability, and a significant proportion of these individuals experience limitations with the movement of their upper limbs [1]. In other words, tasks such as reaching for a cup, opening a door, or picking up dropped objects present a serious challenge to this group of population and require external assistance.

In the United States alone, about 12.2% of the population face mobility issues such as climbing up the stairs or walking, which often correlates with upper body control and reach [2]. Additionally, studies in rehabilitation engineering have shown that among individuals with spinal cord injury, over 60% report difficulty in performing daily activities independently due to impaired arm and hand function [3].

There are existing technologies and products on the market that provide assistance to these groups of people with impaired upper limbs. Although they provide partial independence, they often come with trade-offs. Prosthetics can be expensive - the cost ranges between $10,000 and $100,000 depending on the complexity - and may require invasive surgery or months of physical adaptation.

Consequently, our team recognized the need for a responsive, intuitive, and accessible assistive device that could reduce dependence on caretakers while restoring some degree of autonomy to make both parties' lives easier. To address these issues, we propose Helping Hand: an intelligent, assistive robotic arm designed to empower individuals with arm and hand mobility impairments by enabling them to perform essential reaching and grasping tasks through intuitive eye-tracking. To maximize the flexibility based on user’s preference and physical capabilities, the proposed input would be either voice command or an accessible gamepad-style controller.

Helping Hand aims to be a low-cost, non-invasive, and user-friendly solution that combines embedded systems, robotics, and computer vision. Our ultimate goal is not only to restore functional independence in everyday interactions but also to reduce reliance on caregivers and improve user’s autonomy and self-confidence.

By leveraging affordable hardware platforms like NVIDIA Jetson Nano, open-source libraries, and accessible sensors, we hope to close the gap between high-tech assistive robotics and the everyday needs of individuals who simply want to take control of their environments again.

III Problem Formulation

Through extensive research and engagement with potential users, it became evident that no existing solution effectively balances affordability, usability, and independence for individuals with upper limb paralysis. While technology exists to restore some motor function, it is often gated behind high costs, complex interfaces, or clinical settings. Everyday environments demand a different kind of solution — one that is portable, adaptable, and easy to control without requiring fine motor input.

We found a particular lack of tools that could assist users in real-time, unstructured settings like a classroom, home, or public space, where quick, intuitive interaction is essential. Moreover, current assistive arms either depend on residual muscle input or joystick-based control, which are not viable for many users with advanced paralysis or neurological conditions.

This gap in accessibility and control inspired the need for a lightweight, wearable robotic system that responds naturally to a user's intention. The problem is not just technical — it is deeply tied to autonomy, dignity, and daily functioning. Our formulation centers on creating a system that is not only functionally capable but practically usable and empowering for those it serves.

IV Analysis

IV.I I/O Analysis

Our system consists of two camera streams as input, one facing the user to record the motion of our eye movement while the other one facing outward to map out the surroundings of the user. The eye tracking camera feed is first fed into an eye tracking algorithm and the raw position of the iris is extracted. This information is fed into a SLAM algorithm with the outward camera feed to pinpoint the interest points we want the robot arm to get to. After extracting the interest point, the data is fed into our path finding algorithm to output joint torque for our PID controller to execute our desired motion.

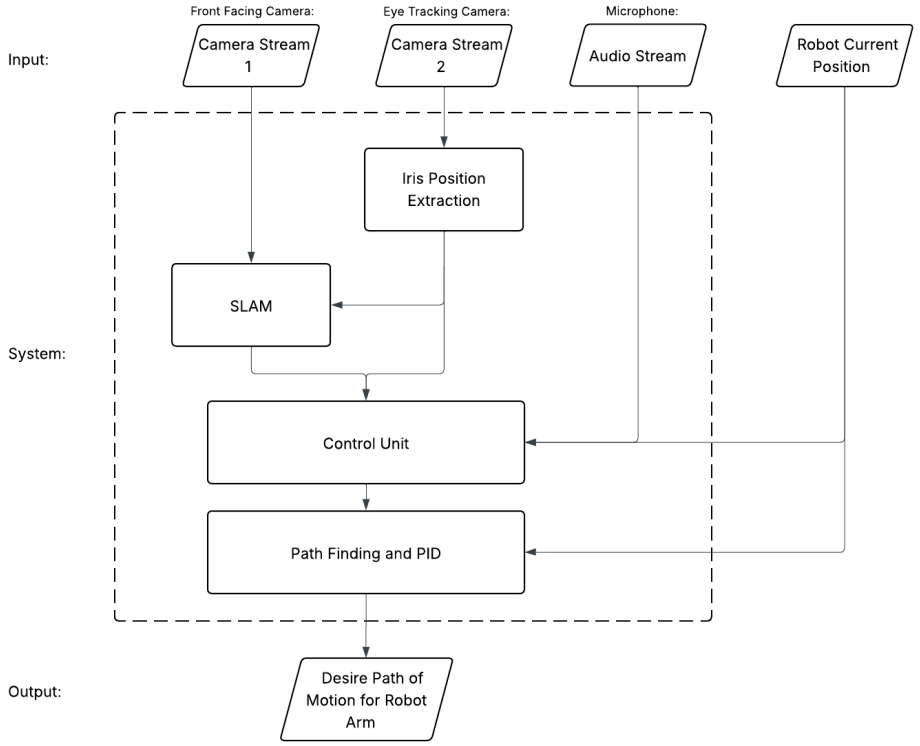


Figure 1: I/O design of the system

IV.II Software Integration

The software of Helping Hand can be broken down into several components as shown in the block diagram below (Figure 2).

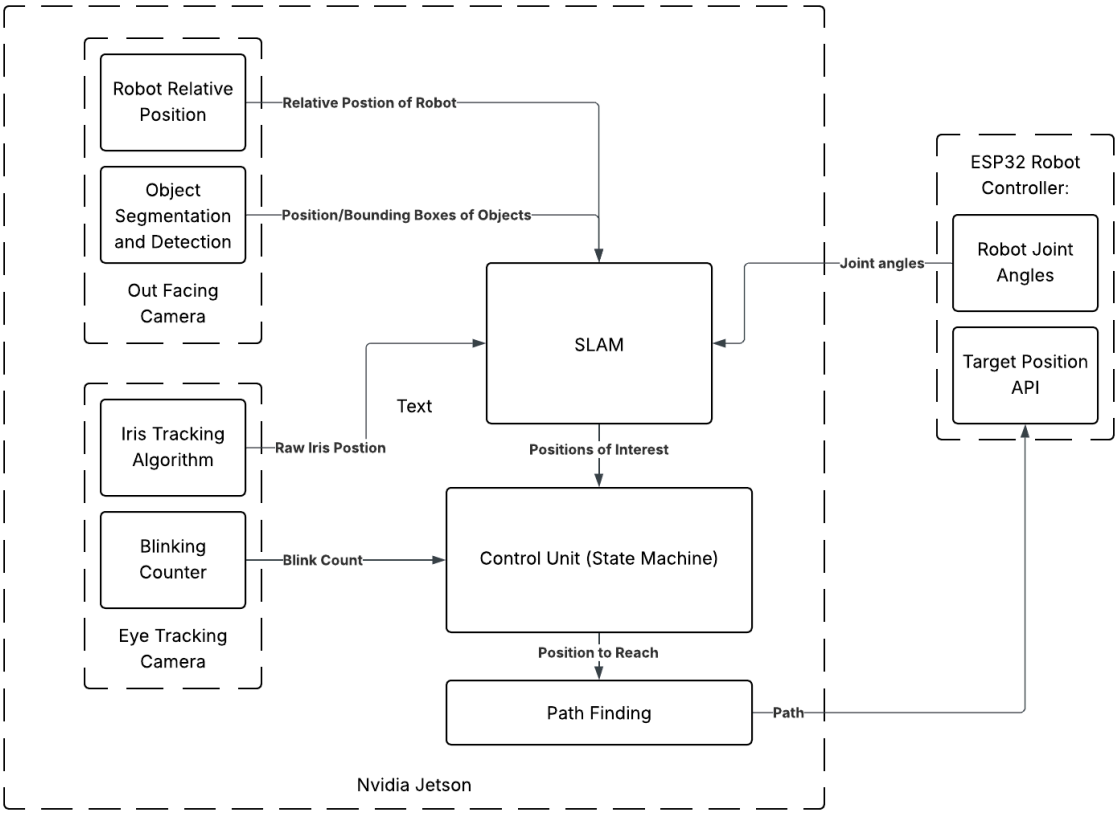


Figure 2: Software design of the system

IV.III Hardware Architecture

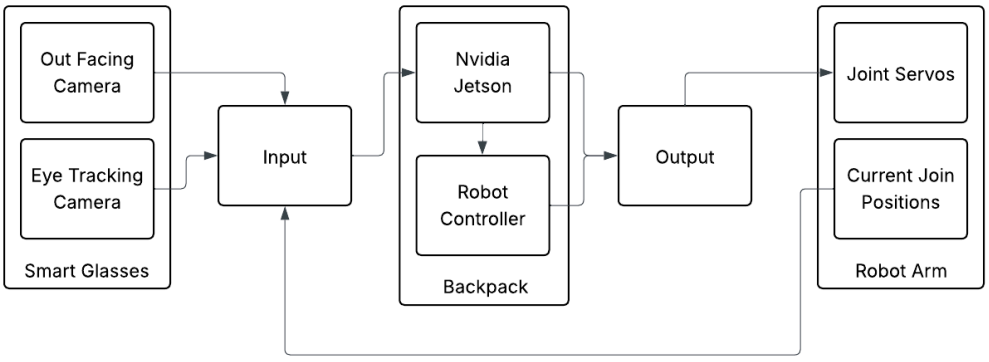


Figure 3: Hardware design of the system

IV.IV Robot Operating System 2 (ROS2)

ROS 2 (Robot Operating System 2) is the second-generation open-source robotics middleware framework that provides a set of libraries and tools for building robot applications, featuring improvements over its predecessor like real-time support, multi-robot communication, and a more robust DDS-based messaging layer. MoveIt 2 is the ROS 2 port of the MoveIt motion planning framework, which provides capabilities for manipulation tasks including kinematic planning, collision detection, trajectory execution, and grasp generation — making it the go-to solution for programming robotic arms and other articulated manipulators in the ROS 2 ecosystem. apriltag_ros is a ROS/ROS 2 wrapper around the AprilTag visual fiducial detection library, enabling robots to detect and estimate the 3D pose of known square fiducial markers in camera images; this is widely used for tasks like object localization, camera calibration, and robot navigation where reliable, low-cost visual landmarks are needed. Together, these three packages form a powerful stack: ROS 2 provides the communication backbone, MoveIt 2 handles motion planning and control, and apriltag_ros supplies visual perception of tagged objects in the environment.

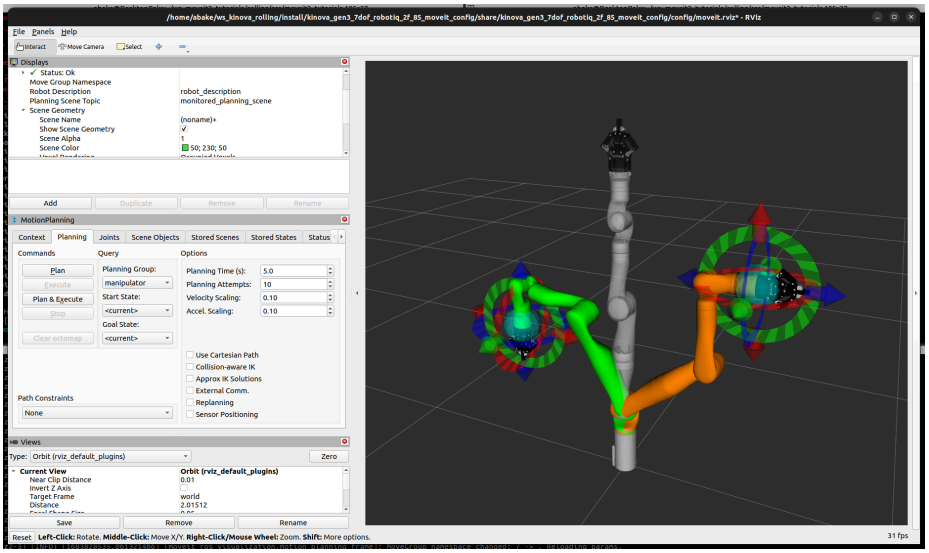


Figure 4: Example simulation through RVIZ2 and Moveit2

V Design Strategies and Approach

V.I Software Components

V.I.I Eye-tracking algorithms

The eye-tracking module is a core part of the Helping Hand system, enabling intuitive, hands-free control over robotic movements. This module will be implemented using real-time computer vision techniques and will run on the Jetson Nano. The system will first perform ROI cropping, isolating the eyes and estimating the pupil center in each frame. From there, we compute the gaze direction vector by analyzing relative pupil movement and iris displacement.

To ensure stability, especially during involuntary blinks or saccades, we will apply filtering algorithms called exponential moving average. A calibration phase will map gaze positions to control regions within the workspace, allowing the user's gaze to translate into directional commands or target object selections. This system will work in tandem with the environment camera to determine both what the user is looking at and where in space the robotic arm should reach.

Our implementation will use Python-based frameworks like OpenCV and Dlib for early prototyping, then move to optimized GPU-accelerated models with TensorRT for real-time performance on Jetson Nano.

V.I.II User Interface

The user interface (UI) is designed for system monitoring, calibration, and optional manual override. It will run on the Jetson Nano, exposed either as a local desktop application or a web-based dashboard accessible via wireless connection. Key features include eye calibration interface: that guides the user through gaze target mapping, live system status, and manual controls that allow the user or caregiver to issue commands via buttons or sliders if needed. It could additionally include voice override logging that displays recognized commands from the voice decoder.

The UI will be built using Python's Tkinter or PyQt for the local version, or React + Flask for the web version. Communication with background processes will use shared memory or WebSockets depending on architecture.

V.I.III Image Recognition

An open-sourced pre-trained model such as MobileNetV2 or an object detection model such as SSD will be used to train the Helping Hand to recognize the specific object that the user's eyes are focusing on. The model will be initially deployed using Tensorflow on a laptop. After tuning and further training until the accuracy rate meets our requirement, it will be

ported to the main board processor. This part of the software development heavily utilizes hardware components, especially the camera, in which its captured footage will be processed frame by frame to detect and classify nearby objects in the environment.

It’s worth noting that the transition from training on a laptop to port on the main board has an additional step. The model trained by Tensorflow from Python will first be converted to the Tensorflow Lite format to reduce its computational footprint, thus reducing the processes and power consumption on the motherboard.

Overall, the integration of visual detection and gaze-based attention mapping will allow the Helping Hand to understand the user's intentional focus, enabling the robotic arm to reach toward and interact with the desired object.

V.I..IV Voice Decoder

To support users with limited eye movement or as an alternate input method, a voice decoder interprets predefined spoken commands. We use a keyword-spotting engine such as Picovoice Porcupine or TensorFlow Lite for Microcontrollers.

Voice input is captured via microphone and processed either locally on the Jetson Nano or on the ESP32 if computation is lightweight. Recognized commands such as “grab,” “release,” or “cancel” are converted into system-level events and prioritized based on context (e.g., override gaze input during confirmation steps).

V.II Hardware Components

V.II.I Robotic Arm

The robotic arm that will be integrated into this project's hardware ecosystem is the WaveShare RoArm-M3-S. This robotic arm features a full 360 degree base, enabling full six degrees of freedom movement and manipulation of the environment within the arm's 1.12 meter diameter of operation. The robotic arm assembly with clamp and associated control circuitry attached comes in at roughly 973 grams, meaning that the arm will be able to be easily supported by an on-body harness or backpack-like mounting system. The arm can also be easily modified to extend its diameter of operation or to reduce total system weight, meaning that this platform is a good mechanical fit for this project.



Figure 5: The RoArm-M3-S robotic arm with attached clamp mount and integrated ESP32 microcontroller board installed [4]

The RoArm-M3-S has its six ST3215 servo motors and singular ST3215-HS servo motor controlled by an ESP32-based microcontroller board within the arm's base assembly. With this ESP32 microcontroller and WaveShare providing a fully open-sourced library of hardware control commands and parameters for the arm, wired or wireless communication will be able to be performed between the robotic arm and the main processing board of the system (V.II.II). This communication between the arm and main processing board can take place over 802.11 b/g/n Wi-Fi, Bluetooth 4, UART serial, USB, or even the ESP32's ESP-NOW protocol. Using the provided ESP32 solution for arm control allows the separation of environment processing command delivery, ensuring that the main processing board will be able to use most of its computational resources for the eye-tracking and environmental sensing portion of the project.

Additionally, the RoArm-M3-S supports the LeRobot software framework, enabling deep learning, training models, and artificial intelligence reinforcement that will enable a higher level of environment interaction and gesture accuracy for the end user.

V.II.II Main Processing Board

The NVIDIA Jetson Nano is the main processing board of our robotic arm, connecting the hardware and software components of our project. The Jetson Orin Nano Super features a six-core ARM Cortex-A78AE processor alongside an NVIDIA Ampere architecture GPU with 1024 CUDA cores for parallel computing and hardware acceleration. Additionally, the board supports a wide range of I/O through USB 2.0/3.0, SPI, I2C, GPIO, UART, wireless communications (through an M.2-E interface and a wireless module), MIPI CSI, Gigabit Ethernet, and both HDMI and DisplayPort video out. The Nano module is also fitted with 8GB of LPDDR5 and was given a 512GB NVMe SSD, ensuring that there will be enough RAM and on-board storage to hold and run the software side of this project.

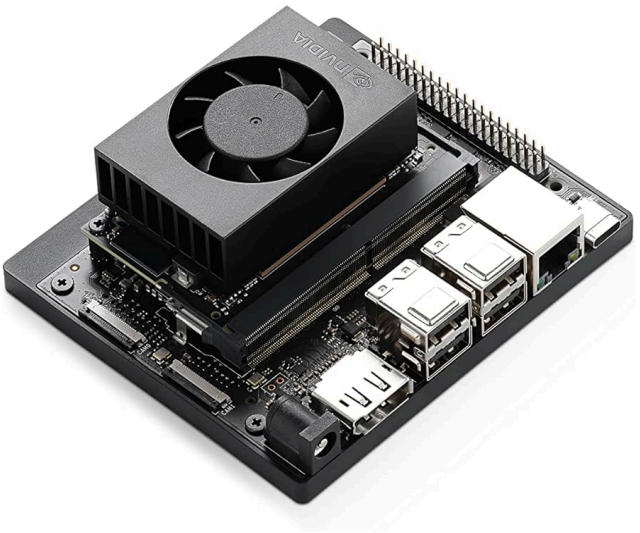


Figure 6: The NVIDIA Jetson Orin Nano Super module installed into a developer kit carrier board.

Due to the Jetson Orin Nano's system-on-chip (SoC) and its integrated Ampere GPU, LeRobot training, models, and deep learning can be run directly on the main compute platform of this project without needing an external laptop or desktop with a discrete NVIDIA GPU to perform training or calibration. This should make spatial calibration, robotic arm training, and per-user setup faster and easier.

V.II.III Cameras and Sensors

Using the Jetson Orin Nano and its carrier board's two integrated MIPI-CSI interface connectors, two cameras will be simultaneously utilized to both perform eye and pupil tracking as well as front-facing environment monitoring and tracking. Both cameras will be

mounted onto a glasses frame-like mounting mechanism that can be worn as glasses or around the head by the end user.

The first camera is a 120 Degree IMX219 CSI-compatible camera with a relatively low resolution but with the ability to send a high number of frames per second (ideally 20-30 FPS minimum) in order to view the pupil and record motions of the eye that will eventually be translated into robotic arm commands. The camera will be mounted at an angle at the edge of the user's peripheral vision to reduce visual obstruction as much as possible. The second camera, a 72 Degree IMX219, is also compatible with the CSI interface and will be mounted onto the headgear mount, but this camera will be facing outwards to simulate user line of sight. This camera will be used to detect objects in the environment and will be used as a positional calculation and calibration device through computer vision software. This camera will prioritize resolution over framerate to get the best possible picture for edge detection and object sensing, but a target framerate of at or above 15 FPS will be ideal for smooth object detection during slight user movement or camera shake.

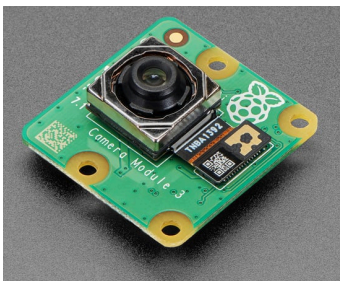
(a)  (b) 

Figure 7: (a) 120 Degree IMX219 CSI Camera (inward) (b) 72 Degree IMX219 CSI Camera (Outward)

For voice commands, a simple USB microphone, SPTSSWET USB-MKF-US-001, is used with the Jetson Orin Nano's USB ports to pick up user commands and translate them into robotic arm or system commands. While the exact microphone chosen does not drastically affect the full design approach of the system, a microphone with high enough audial fidelity (good quality speech pickup, low noise, low background pickup, etc.) and with a relatively low cost is ideal. The microphone could be either mounted directly into the Jetson Nano USB ports or could be mounted somewhere on the robotic arm or on the user camera headgear assembly for better voice pickup.



Figure 8: SPTSSWET USB-MKF-US-001 microphone

V.II.IV System Power, Charging, and Management

For portability, high energy density, and a relatively low price solution, the robotic arm, vision and voice systems, and NVIDIA Jetson Orin Nano main processing board will be powered by a 12V, 9-12Ah battery unit. The battery will either be a lead-acid unit or a lithium ion phosphate (LiFePO4) series and parallel assembly of many smaller 3V cells. The 12V battery voltage rail will be used to power the robotic arm directly, and this rail will be stepped up or down to 12V with a buck-boost converter board within the main processing board assembly to supply the Jetson Orin Nano with its required 12V input. Both battery solutions will have enough current delivery headroom to supply both the robotic arm and main processing board during current draw spikes during sudden movements or the start of computation on the Jetson Orin Nano's SoC.

For battery charging, a battery charging system will be created for the specific battery solution that is chosen for the final design. The batteries will accept both AC 120V wall power in as well as DC in through either an additional DC power source (such as a laptop charger) or a solar panel assembly if a user desires mobile or off grid charging. The DC in port or jack will support DC 5V up to DC 24V for a wide range of DC charging capability and will be stepped up or down as needed with a buck-boost converter circuit within the overall system assembly. If AC charging is used, the 120V AC will be rectified, stepped down, and smoothed into the required arm and compute platform voltage input level. The system, whether receiving AC or DC, will monitor battery charge current, battery temperature, and current cell/pack voltage to ensure that the batteries or battery cells are balanced, cool, and are not overcharged.

VI Parts and Implementation

VI.I Eye Tracking System

VI.I.I Eye Tracking Module Design

The eye tracking system is designed as a vision-based module that detects the user's pupil, estimates pupil gaze direction, and translates it into a real-time control signal for the robotic arm. The final design of the eye tracking module consists of two cameras, one facing outward (back camera) and one facing inward (front camera). The front camera is used for eye tracking and the back camera is used for environmental sensing. The design enables a stable reference frame between the user's eye and the surrounding environment, which can also reduce the dependency on external head tracking systems deployed in traditional facial tracking systems such as MediaPipe.

The front-facing camera captures image frames of the user's eye region (ROI), which is then processed through a series of pipelines including preprocessing, pupil detection, feature extraction, normalization, and calibration. The back camera operates simultaneously to detect visible AprilTages in the environment, and provide a cursor that acts as the spatial reference point of the user's gaze for interaction. By integrating the postprocessed information from these two data streams, the eye tracking system is capable of understanding user intent by combining gaze direction and pupil coordinates with contextual scene information.

VI.I.I Image Preprocessing

The eye tracking pipeline begins with continuous frame acquisition from the front facing CSI camera mounted onto the glass frame using OpenCV-based interfaces. Each captured frame is first converted to grayscale. This process helps reduce the computational complexity while improving robustness to color variation. The equation for computing grayscale intensity is

$$I_{gray} = 0.299R + 0.587G + 0.114B$$

where R stands for red, G stands for green, and B stands for blue.

Another technique called Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to the grayscale image during this preprocessing stage. The purpose of applying this technique is to enhance contrast and improve pupil visibility under varying lighting conditions. All image preprocesses are applied to the two cropped frames around the user's eyes, known as region of interest (ROI). This region of interest was predefined by the user during the calibration stage, to ensure the system can focus most of its resource on processing pupil and gaze information without distractions from the environment.

VI.I.II Pupil Detection and Feature Extraction

Within the ROI, the system performs pupil detection using image processing techniques based on grayscaled frames. The primary method involves thresholding and contour detection to isolate dark regions corresponding to the pupil. The binary segmentation is defined as

$$B(x,y) = \begin{cases} 1 & I(x,y) < T \\ 0 & \text{otherwise} \end{cases}$$

which the largest valid contour extracted from the binary image is selected as the pupil candidate. An eclipse is then fitted to the contour using a least-squares approach, providing an estimate of the pupil center (x_p, y_p). The detected pupil center serves as the primary feature for gaze estimation.

VI.I.III Normalization and Coordinate Transformation

The system applies geometric normalization to the detected pupil coordinates. This process ensures small movements or vibration won't cause the tracking system to lose focus, as the pupil position is expressed relative to a fixed reference region rather than absolute pixel coordinates. The equation for computing normalization is the following

$$g_x = \frac{x_p - x_{ROI,left}}{w_{ROI}}, \quad g_y = \frac{y_p - y_{ROI,top}}{h_{ROI}}$$

This transformation converts pixel coordinates into dimensionless values in the range [0,1], making the features invariant to resolution, scale, and minor positional shifts.

As a result, minor displacements of the eye or camera do not introduce significant inaccuracies in gaze estimation, allowing the system to maintain stable and consistent tracking performance.

VI.I.IV Calibration and Gaze Mapping

A calibration procedure is used to map normalized eye features to spatial coordinates in the environment. During calibration, the user fixates on known targets such as AprilTags, and corresponding feature-target pairs are recorded as:

$$(g_x, g_y) \rightarrow (x_{target}, Y_{target})$$

A linear model is then used:

$$x = ag_x + bg_y + c, \quad y = dg_x + eg_y + f$$

The model parameters are obtained using least-squares optimization:

$$\theta = (X^T X)^{-1} X^T Y$$

This mapping enables the system to convert real-time gaze features into predicted spatial positions for interaction.

VI.I.V Blink Detection and Control Logic

In addition to gaze estimation, the system incorporates blink detection as a discrete control input. Eye closure is detected using geometric features such as Eye Aspect Ratio (EAR), defined by the equation:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

A sequence of blink events is monitored over time. A control command is triggered when three blinks are detected within a five-second window. This mechanism provides an intuitive and reliable method for user input without requiring additional hardware.

VI.II Voice Control System

VI.II.I Voice Command Module Design

The voice control system is designed as a complementary input modality that allows the user to issue discrete commands to the robotic arm through spoken keywords. The module is implemented entirely on-device using the Vosk speech recognition toolkit, which is an offline engine built on the Kaldi ASR framework. Running recognition locally avoids network latency, preserves user privacy, and ensures the system remains functional without an internet connection.

The module runs as a ROS 2 node that continuously captures audio from a microphone, transcribes it through a pre-trained Vosk acoustic model, matches the transcription against a set of predefined keywords, and publishes the corresponding command to the finite state machine for execution by the robotic arm.

VI.II.I Voice Command Module Design

Audio is captured from a USB microphone using the PyAudio library as single-channel 16-bit PCM samples at a 16 kHz sampling rate, which matches the input rate expected by the Vosk model. Samples are read in fixed blocks of 4000 frames (~250 ms) to balance recognition latency against decoder stability.

VI.III.I Speech Recognition with Vosk

The preprocessed audio is streamed into a Vosk KaldiRecognizer initialized with the small pre-trained English model (vosk-model-small-en-us-0.15). Internally, Vosk extracts MFCC features from each audio block, passes them through a DNN acoustic model to produce per-frame phonetic posteriors, and decodes the most likely word sequence using a weighted finite-state transducer (WFST).

To improve accuracy on our narrow command set, the recognizer is initialized with a grammar argument that restricts the decoder to a predefined vocabulary. This pruning of the search space reduces both the recognition error rate and the computational cost of decoding on embedded hardware.

VI.II.IV Keyword Matching and Command Dispatch

Finalized transcriptions are normalized (lowercased, whitespace-stripped) and passed through a lookup table that maps spoken phrases onto discrete robot commands. Multiple synonyms are aliased to the same command ("grab," "pick up," and "retrieve" all trigger a retrieve action) so that the system accommodates natural variation in how users phrase their intent.

Once a keyword is matched, the resulting command is forwarded to a ROS 2 node that publishes it onto the same output stream used by the eye tracking pipeline, so voice commands and eye-based commands are semantically equivalent from the perspective of the downstream finite state machine. This unified command channel allows the user to freely switch between modalities, and makes the voice interface a drop-in alternative to the triple-blink gesture without requiring any changes to the FSM or manipulation logic.

VI.III RoArm-M3-Pro Vision-Guided Manipulation System

Our system leverages the RoArm-M3-Pro robotic arm equipped with an eye-in-hand USB camera to perform real-time object detection and autonomous grasping. The implementation pipeline consists of three stages: camera calibration, fiducial-based object localization via AprilTag detection, and position-based visual servoing (PBVS) through MoveIt 2 for closed-loop control of the manipulator.

VI.III.I Camera Calibration and Intrinsic Parameter Estimation

A USB camera was rigidly mounted to the end effector of the RoArm-M3-Pro, forming an eye-in-hand configuration. Before any pose estimation could be performed, we calibrated the camera to obtain its intrinsic parameters using a checkerboard calibration target and the standard pinhole camera model. The projection of a 3D world point $P = (X, Y, Z)^T$ onto pixel coordinates $p = (u, v)^T$ is governed by:

$$s \cdot [u, v, 1]^T = \mathbf{K} \cdot [\mathbf{R} \mid \mathbf{t}] \cdot [X, Y, Z, 1]^T$$

where $\mathbf{K}$ is the 3×3 intrinsic matrix containing the focal lengths (f_x, f_y) and principal point (c_x, c_y), $\mathbf{R}$ and $\mathbf{t}$ are the extrinsic rotation and translation, and s is a scale factor. We solved for $\mathbf{K}$ and the lens distortion coefficients (k_1, k_2, p_1, p_2, k_3) using Zhang's method over multiple checkerboard poses, minimizing the reprojection error across all detected corner points. The resulting intrinsics were published as a ROS 2 camera_info topic for downstream consumption.

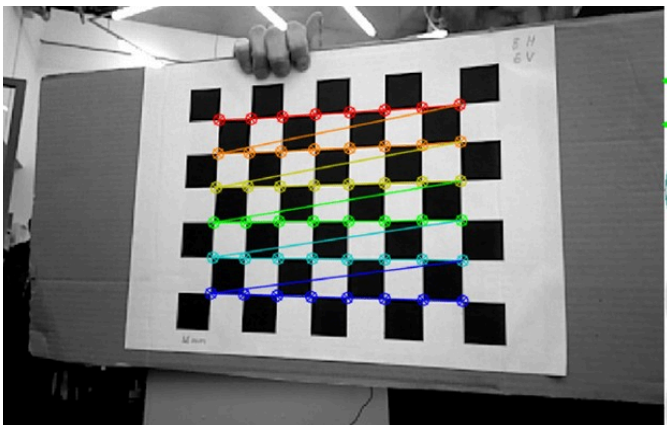


Figure 10: Camera Calibration Checkerboard

VI.III.II AprilTag-Based Object Localization

With calibrated intrinsics, we employed the apriltag_ros package to detect fiducial markers affixed to target objects. The AprilTag detector identifies the four corner points of each tag in the image and, given the known physical tag size and the camera intrinsic matrix **K**, solves the Perspective-n-Point (PnP) problem to recover the 6-DOF pose of the tag relative to the camera frame. This yields a homogeneous transformation ${}^{c}T_{tag} \in SE(3)$ representing the position and orientation of the tag in the camera's coordinate frame. Since the camera is rigidly attached to the end effector, the pose of the object in the robot's base frame is obtained through the kinematic chain:

$$ {}^{b}T_{tag} = {}^{b}T_{ee} \cdot {}^{ee}T_{cam} \cdot {}^{c}T_{tag} $$

where ${}^{b}T_{ee}$ is the forward kinematic transform from base to end effector (computed from joint states), and ${}^{ee}T_{cam}$ is the fixed hand-eye calibration transform between the end effector and camera frames.

VI.III.III Position-Based Visual Servoing with MoveIt 2

With the target object localized in the base frame, we implemented a position-based visual servoing (PBVS) loop to guide the arm toward the object. At each control iteration, the pose error between the current end-effector pose and the desired grasp pose is computed as:

$$ \mathbf{e} = {}^{b}T_{tag}^{-1} \cdot {}^{b}T_{ee} $$

from which we extract a translational error $\Delta\mathbf{p} = (\Delta x, \Delta y, \Delta z)^T$ and a rotational error $\Delta\boldsymbol{\theta}$ (extracted via the axis-angle representation of the rotation component). MoveIt 2's motion planning pipeline then generates a collision-free trajectory that drives $\mathbf{e} \rightarrow \mathbf{I}$ (the identity transform, i.e., zero error). Rather than planning a single open-loop trajectory, we re-acquire the tag pose at each servo step, recompute the error, and issue incremental waypoint commands through MoveIt 2's servo interface. This closed-loop behavior compensates for minor calibration inaccuracies and any movement of the target. Once the translational and

rotational errors fall below defined thresholds ($\|\Delta\mathbf{p}\| < \varepsilon_p$ and $\|\Delta\boldsymbol{\theta}\| < \varepsilon_\theta$), the system triggers the grasp sequence, commanding the gripper to close and secure the object.

This architecture provides a robust, real-time pick pipeline: the camera continuously streams frames, apriltag_ros publishes tag poses at frame rate, and MoveIt 2 servoing iteratively corrects the arm's trajectory until the grasp is achieved. The entire stack runs within the ROS 2 framework, with each component communicating via standard topics and transforms (TF2), ensuring modularity and ease of integration.

VI.IV Finite State Machine (FSM)

The high-level behavior of the system is governed by a finite state machine (FSM) implemented in Python, which orchestrates the interaction between the user's intent — communicated through eye blinks — and the robot's manipulation actions. The FSM provides a clean, interpretable control architecture that transitions the system through three discrete operational modes based on real-time user input.

VI.IV.I State Definitions

The FSM consists of three states. **State A (Observation)** is the default idle state in which the system passively monitors the user's gaze direction and blink activity. The camera feed is processed continuously to track the user's eye state and determine which object, if any, the user is fixating on. **State B (Retrieve)** is entered when the user issues a deliberate command; in this state the robot plans and executes a grasp on the designated object, picks it up, and delivers it to the user. **State C (Return)** is triggered once the object has been delivered and the user signals that they are finished; the robot then transports the object back to its original location, places it down, and transitions back to the observation state.

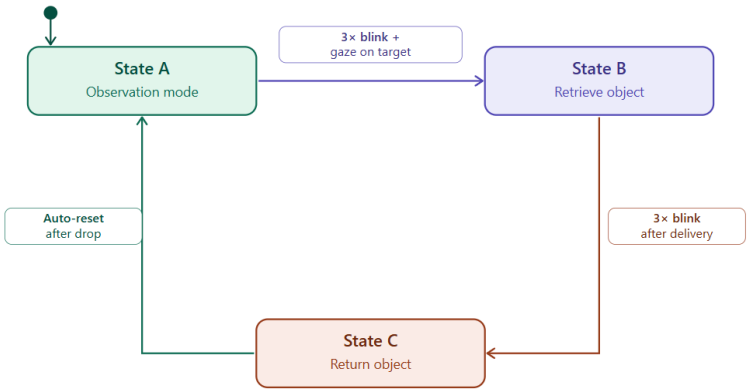


Figure 11: FSM Diagram

VI.IV.II Implementation

The FSM was implemented using a straightforward Python class with an enumerated state variable and a single update method called on each control loop iteration. The simplicity of this architecture is intentional. Each state maps directly to a well-defined robot behavior, and the sole input modality — the triple-blink gesture — eliminates the need for any physical controller or voice interface. The FSM runs as a ROS 2 node that subscribes to the blink detector and gaze tracker topics, queries the AprilTag localizer for target pose information, and issues manipulation commands to MoveIt 2 through the servoing pipeline described previously. This design keeps the control logic decoupled from the perception and planning layers, making it straightforward to extend — for example, by adding additional states for multi-object tasks or integrating alternative command modalities in the future.

VI.V Hardware

VI.V.I Eye Tracking Glasses Design

For this project, early on challenges arose regarding finding the perfect camera height and angle in order to properly calibrate our eye tracking software. To circumvent this problem, we decided to fabricate our own set of eye tracking headwear using the in lab FDM printer. The resulting hardware was a band style design that utilizes a 360 degrees swinging arm to allow for any angle we could possibly need. We additionally included a central camera mount that enabled our system to see whatever was directly in front of the user.

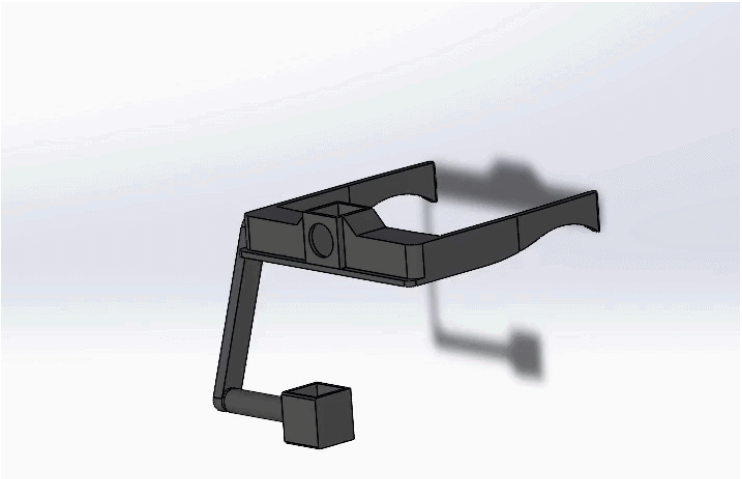


Figure 12: Mechanical Design of the Eye Tracking Glasses

VI.V.II Eletronic Case Design

As the amount of boards we were using started to mount, we quickly realized that a storage solution was needed in order to elegantly compact all our electronics into a centralized point. The answer to this problem ended up being a 3d printed two layer modular box that allowed

for easy combination of our two boards. To make these two sections easily attachable, we used a simple sliding mount mechanism paired with tiny magnets that allowed the user to quickly slide and satisfyingly click the two boards together. We also made sure to include a simple hook in the back to allow for attachment to wheelchairs of all different shapes and sizes.

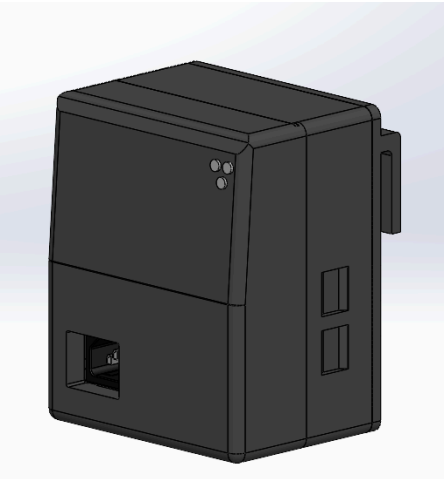


Figure 13: Mechanical Design of the Modular Electronic Case

VI.V.III Gripper Design

The custom gripper for the Helping Hand system was designed in CAD and 3D printed to interface directly with the RoArm-M3-Pro's end effector. The gripper features a dual-finger clamp geometry with two symmetrical, rounded gripping surfaces that converge to securely hold objects such as books and similarly shaped items. The open-loop finger profile provides sufficient clearance for the gripper to approach and enclose target objects without collision, while the curved inner surfaces distribute contact force evenly to reduce slippage during retrieval and transport. At the base, a flat mounting plate with multiple screw holes allows the gripper to be rigidly fastened to the arm's final servo linkage, ensuring a stable mechanical connection that does not introduce unwanted compliance during grasping operations. The part was fabricated using PLA filament on a standard FDM 3D printer, allowing for rapid iteration and low-cost prototyping as the team refined the finger spacing, wall thickness, and approach geometry through repeated grasp testing with the AprilTag-guided visual servoing pipeline.

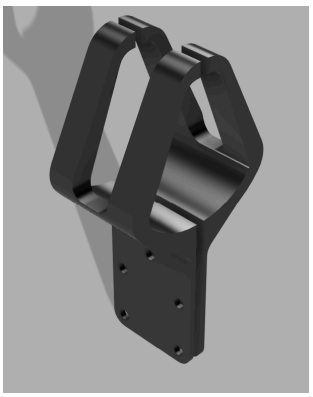


Figure 14: Mechanical Design of the Robotic Gripper

VI.V.IV Camera Mount Design

The camera mount was custom designed in CAD and 3D printed to securely attach the USB camera to the robotic arm's end effector servo. The mount features a dual-slot geometry with two rectangular openings — the larger upper slot accommodates the USB camera body through a press-fit interface that holds the camera snugly in place without requiring adhesive, while the lower slot provides clearance for the camera's USB cable routing. A screw hole positioned at the top of the mount allows the user to tighten the fit and lock the camera firmly into position, preventing any loosening or vibration-induced shifting during arm movement that could compromise the accuracy of the AprilTag detection and visual servoing pipeline. The flat rear plate includes mounting holes that align with the servo horn or bracket on the RoArm-M3-Pro's end effector, ensuring a rigid and repeatable attachment point so that the hand-eye calibration transform between the camera and end effector frames remains consistent across sessions. This simple yet effective design enabled rapid assembly and disassembly during development and testing, while maintaining the precise camera alignment critical for reliable pose estimation and closed-loop grasping.

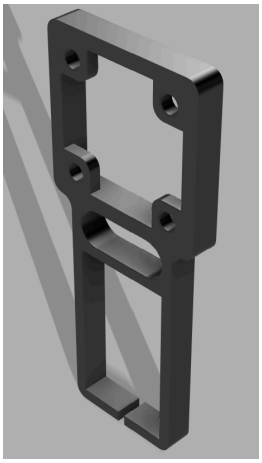


Figure 14: Mechanical Design of the Camera Mount

VI.V Battery and Power System

VI.V.I Power System Introduction

As the system is intended to be a portable solution that is mounted to a wheelchair or motorized chair, a power conversion and delivery board was designed, produced, and tested in order to provide a complete power system solution for installation within the Helping Hand system.

The final designed system is powered off of a 12V 9Ah sealed lead-acid battery (SLA), and this battery is charged from the power board when the user plugs the system into a wall outlet either at home or out in a shared public space. The power board accepts North American AC wall power as an input (designed for 90-130VRMS AC input), and the board features an AC EMI stage, a rectification stage, and a custom DCM flyback converter stage complete with a hand-wound flyback inductor. The flyback stage provides 14.4V at 2A to charge the battery, and to ensure that the battery outputs a stable 12V no matter the state of charge, the power board features a 12V 6A buck-boost stage in order to supply power to the host computer and robotic arm.

VI.V.II AC Input and EMI Stage

The AC input is supplied to the board through a standard IEC 302 C14 connector (soldered to the PCB) and external matching cable, which is the connector and cable pair that is often seen used with desktop PC power supplies and TVs. This AC input first travels through an AC EMI stage, featuring a 2.2mH common-mode choke and parallel 100nF X2 and 2.2nF Y2 capacitors to filter noise and transient spikes across line and neutral. For safety and protection, the AC input first travels through a 1A in-line slow blow fuse, and the EMI stage also features two 1MΩ bleed resistors and a 10Ω NTC thermistor to discharge large capacitances when the board is off and to limit inrush current respectively.

After the EMI stage, the filtered and clean AC input is fed into a full-bridge rectifier IC, specifically the ABS10-HF, and the output absolute value sine wave signal is then connected to a large 250V 120uF capacitor in order to create a relatively stable 165-175V DC rail even under load.

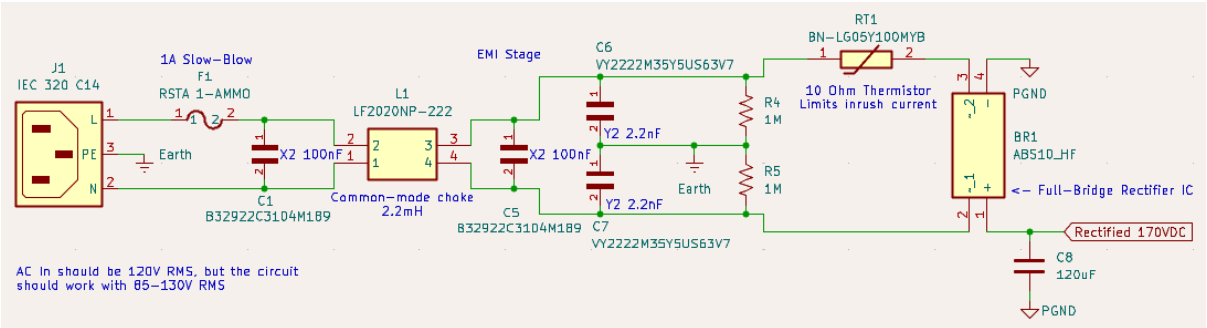


Figure 15: AC Input, EMI, and Rectification Stage Schematic

VI.V.III DCM Flyback Converter Stage

The roughly 170VDC rail is connected as the input to the DCM flyback converter stage through a CLC network of two 10uF capacitors in parallel with a series 470uH inductor to account for the large spikes in drawn current when the Jetson Orin Nano or robotic arm goes into an intense period of computation or movement respectively. This CLC network operates as a very stable source of current, which is beneficial for the stability of the flyback converter's operation and output. This CLC network then is connected to the rest of the flyback converter as the high-voltage (HV) rail. On the primary side, the Texas Instruments UCC28730DR primary side flyback controller IC was chosen for this design, as the IC allows the circuit to operate in DCM through a quasi-resonant valley switching approach and also interfaces with another TI IC on the secondary side for wake-up and secondary/auxiliary winding sensing without a traditional optocoupler. Additionally, the UCC28730DR features an internal high voltage step-down charge pump to initially drive the primary power MOSFET of the system at start-up, which allows the HV line to directly supply the IC during start up and reduces wasted power when compared to a traditional resistor divider or current limiter network. Discontinuous conduction mode (DCM) was chosen for this system, as operating in DCM results in a relatively high power factor of the overall converter (which raises efficiency) and also allows for a smaller primary magnetizing inductance, making the creation of a flyback inductor simpler and more flexible.

On the secondary side of the flyback stage, the Texas Instruments UCC24650DBVR secondary voltage monitor and wake-up IC was chosen in conjunction with the primary side controller IC. This IC is able to sense the auxiliary winding of the flyback inverter and accordingly enables the output rail through an RC feedback network, which stabilizes the switching action of the flyback stage and allows for a varying duty cycle on the primary side based on current load and rectified input voltage.

The primary and secondary sides of the flyback stage feature a significant amount of feedback and compensation circuitry, and both sides also feature the needed snubber diodes, reverse polarity protection diodes, and Zener diodes for clamping in the event of a huge transient voltage spike or incorrect input voltage.

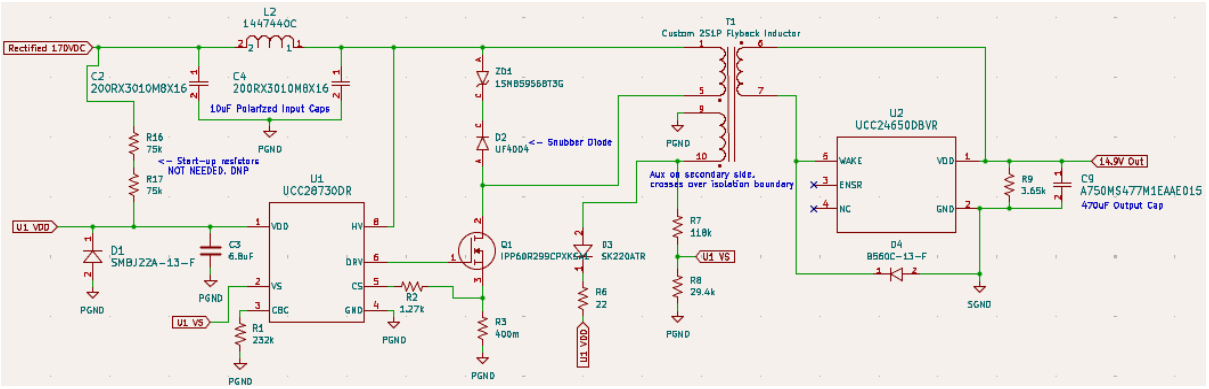


Figure 16: Flyback Converter Stage Schematic

For the flyback inductor, a custom solution was developed from scratch. The flyback inductor features a primary to secondary turns ratio of 30:6, and the flyback inductor was also fitted with an eight-turn auxiliary winding to provide a bias voltage for the primary side controller IC once the IC's internal charge pump is unable to provide enough power to continuously switch the power MOSFET. The primary winding was wound with three parallel strands of 25AWG magnetic copper wire and is separated into to half winding portions, with one half (15 turns) on the very surface of the bobbin and the other half (other 15 turns) on the outside of the secondary and auxiliary winding layer. The secondary was wound with one strand of 20AWG magnetic copper wire, and the auxiliary winding was wound with four parallel strands of 28AWG magnetic copper wire. The primary winding has an opposite rotary winding direction (ex. CW vs. CCW) when compared to the secondary and auxiliary windings, which maintain the dot polarity seen in the schematic above. All layers are separated from each other properly with three layers of Kapton tape to ensure proper magnetic isolation between windings, and the edges of the bobbin are also wrapped with standard electrical tape to provide short circuit prevention in the case of the enamel on the enameled magnetic wire failing due to heat or age. For this flyback inductor, the TDK B66208B1110T001 coil bobbin was used alongside two TDK B66317G0000X187 N87 E-type ferrite core halves. The core halves are secured with two matching TDK yoke clamps and additionally have four layers of electrical tape wrapped tightly around the outer perimeter to ensure a tight and secure inductor build. Before being installed, the cores were gapped through a trial and error process with an LCR meter through layers of Kapton tape being inserted in between the two joining middle faces of the E-type cores, and the primary inductance of 319uH was reached with two layers of standard Kapton tape as a gapping barrier.

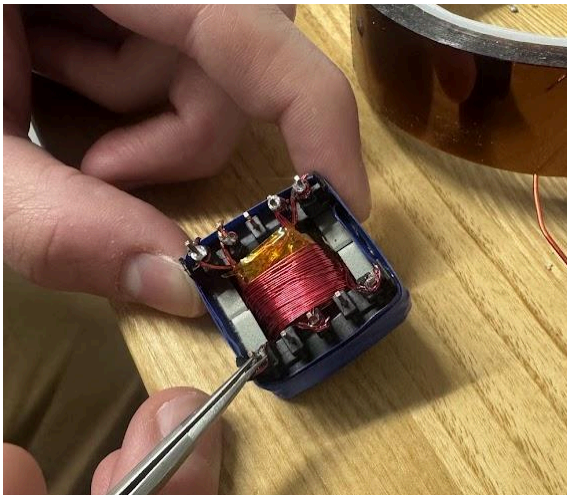


Figure 17: Final Stages of Custom Flyback Inductor Creation

VI.V.IV Buck-boost Stage

Finally, the power board features a 12V 6A buck-boost stage to provide the Jetson Orin Nano computer and robotic arm with a stable 12V input rail regardless of the state of charge of the SLA battery. For the buck-boost stage, the Texas Instruments TPS55289RYQR integrated switch buck-boost controller IC was chosen due to its small footprint, simple passive requirements, and high current output at 12V. Alongside passive feedback and compensation networks, the TPS55289RYQR switches an internal MOSFET to drive a 2.2uH inductor within the buck-boost topology.

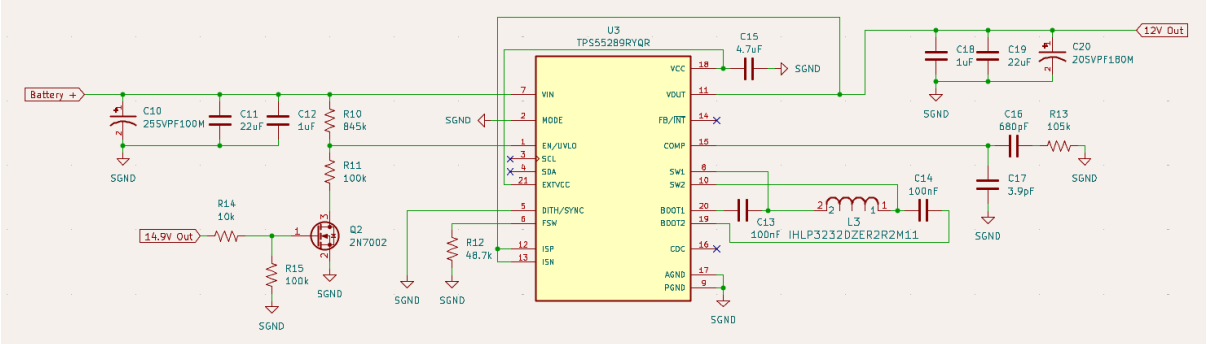


Figure 18: Buck-boost Stage Schematic

VI.V.V PCB Design and Assembly

In terms of PCB design considerations, the board was developed with proper isolation barriers between different stages and ground reference points in mind as a high priority. Earth ground, primary ground, and secondary ground are all isolated by at least 6mm gaps of no copper, whether that be traces or copper planes, which follows proper IEEE and United States safety recommendation guidelines for AC to DC consumer power electronics. Additionally, the board was designed with proper trace widths in mind, and all high-current paths are properly sized and oriented for minimal heat buildup and noise crosstalk. The board also features a large number of thermal vias across different ground reference planes to ensure proper heat dissipation and thermal relief.

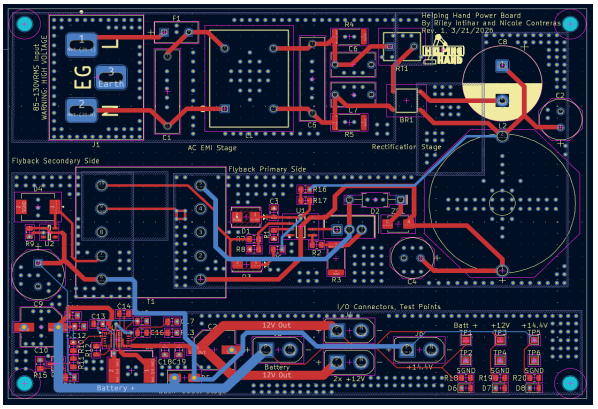


Figure 19: Power Conversion and Delivery Board PCB Layout

The boards were ordered from JLCPCB, an Asian PCB fabrication company, and due to tariffs and a large number of custom components, the board was soldered entirely by hand with components manually purchased from DigiKey and Mouser instead of being assembled by a manufacturer such as JLCPCB. While this was time-consuming and is inefficient for large deployments of this system, in an actual manufacturing environment where hundreds of these boards are to be produced, the cost of full professional pick-and-place assembly would be much cheaper and more viable.

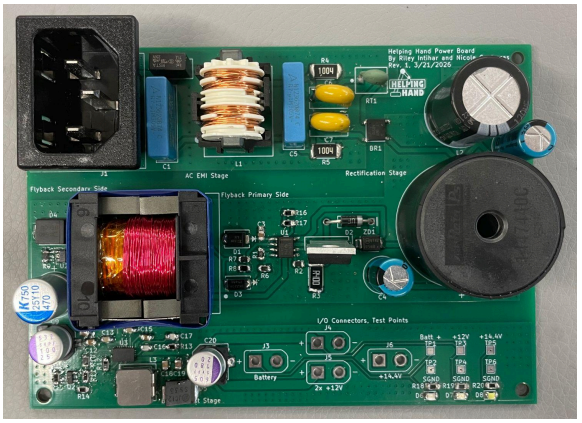


Figure 20: Assembled Power Board

VII Testing Procedures and Design Validation

VII.I Eye Tracking Testing

To ensure the reliability and functionality of the entire eye tracking system, three sets of tests were conducted at different stages of development. The first set of tests focused on camera connection and basic image acquisition, verifying that both CSI cameras were properly interfaced with the Jetson Nano and capable of capturing stable frames. The second stage evaluated the core detection components independently, including pupil tracking from the front facing camera and AprilTag detection from the back camera. The final stage tested the complete pipeline, incorporating calibration and gaze mapping to assess the system's ability to translate eye movement into spatial interaction. Together, these stages provided a structured validation process, allowing each level of pipeline to be tested and evaluated in isolation to quickly identify and diagnose potential issues before full system integration.

VII.I.I Camera Connection Test

This test was implemented at the early stage of developing the eye tracking system. The test was designed to verify that the camera is connected to the correct port on the Jetson Nano and the field of view captured by the front facing CSI camera is wide enough to track both eyes. The captured image frames did not display any error or latency issues. The test also verified that frames could be acquired continuously in real time, ensuring stable communication between the camera drivers and the software pipeline.

VI.I.II Pupil Tracking and AprilTag Detection Test

This test was implemented after the suitable open-sourced pupil tracking and April detection libraries were found and incorporated into the eye tracking system. This stage of testing validated the core detection components of the system: pupil tracking from the inward-facing camera, and AprilTag detection from the outward-facing camera. Gaze mapping has not yet been implemented. This feature will be tested and validated in the next stage of testing.

For pupil tracking, the system applied preprocessing and region-of-interest cropping (ROI) to isolate the eye area before performing pupil detection. The test verified that the detected pupil center remained stable and responsive to eye movement within the ROI. This part of the test confirmed that the system could detect pupil and movement continuously at different frames, with certain unresponsive tracking frames when the eyes are slightly off the center of the ROI.

In parallel, the out-ward facing camera was tested for its ability to detect and track AprilTags in real time. The system was evaluated based on detection responsiveness, frame rate consistency, and the ability to identify multiple tags found within the frame. The results confirmed that the camera was able to detect IDs, providing the necessary spatial reference for future gaze-based interaction.

VI.I.III Calibration and Gaze Mapping Test

The final stage of testing focused on the calibration process and the mapping of gaze features to environmental coordinates. During calibration, the user was instructed to fixate on known targets either on a board or digital display while the system recorded corresponding pupil features and target positions. These data points were used as reference points to map normalized pupil coordinates to spatial locations in the scene.

Testing results showed that the system was able to generate gaze estimates and respond to user input in a functional manner, enabling basic interaction with detected targets. However, the system did not achieve fully satisfactory precision. During validation, some level of jitter was observed in the gaze output, even when the user maintained steady fixation on a single target. This instability is primarily attributed to insufficient data and limitations in detection accuracy, which introduce small errors that are amplified during the mapping process. As a result, while the calibration and gaze mapping is operational, further optimization is required to improve stability and achieve higher precision for reliable control.

VII.II Robot Arm Testing

The validation of the RoArm-M3-Pro vision-guided manipulation system was conducted incrementally, isolating each subsystem before integrating components into the full pipeline. This staged approach allowed us to identify and resolve failures at each layer independently, ensuring that downstream modules operated on verified inputs.

VII.II.I AprilTag Localization Verification

The first stage of testing focused on confirming the accuracy and stability of the AprilTag pose estimation pipeline. An AprilTag marker was held in front of the end-effector-mounted camera and moved through a range of positions and orientations while the output of the apriltag_ros node was monitored in RViz. The published TF frame corresponding to the detected tag was visualized in real time against the robot's URDF model, allowing us to verify that the estimated 3D pose tracked the physical marker smoothly and without significant jitter. We checked for consistent behavior across varying distances, approach angles, and lighting conditions. Particular attention was paid to the transform chain ${}^{b}T_{tag} = {}^{b}T_{ee} \cdot {}^{ee}T_{cam} \cdot {}^{c}T_{tag}$ to ensure that the tag's pose was correctly projected into the robot's base frame. Any discrepancies at this stage — such as a reflected axis or an incorrect hand-eye calibration offset — would propagate through the entire system, making this a critical validation gate.

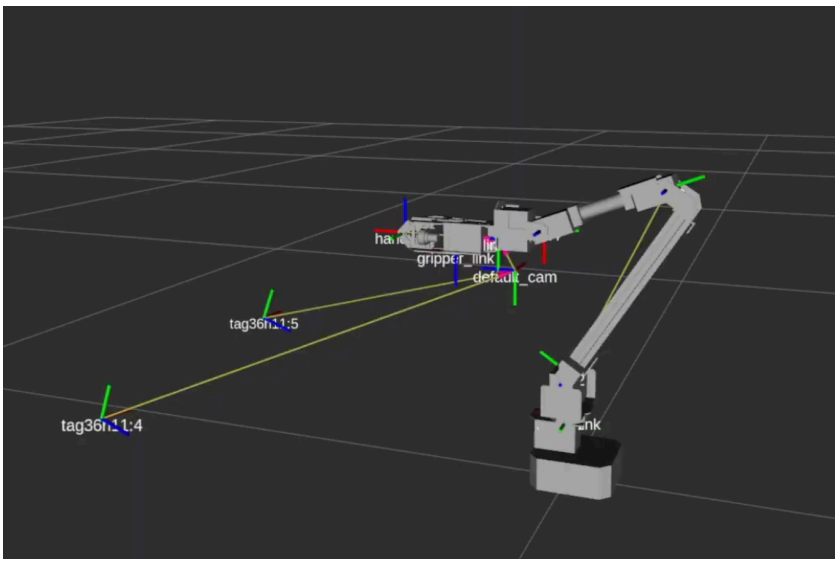


Figure 21: Real time Pose Estimation of April Tag in RViz

VII.II.II Visual Servoing in Simulation

With tag localization confirmed, we proceeded to test the position-based visual servoing (PBVS) loop in isolation using RViz and MoveIt 2's simulated planning environment. A virtual tag pose was published at a fixed location, and the servoing controller was commanded to drive the end effector toward it. We verified that the pose error $e = {}^{b}T_{tag}^{-1} \cdot {}^{b}T_{ee}$ converged monotonically toward the identity transform, and that the planned trajectories were smooth, collision-free, and respected the joint limits of the RoArm-M3-Pro. This simulation phase allowed us to tune the servo loop gains and waypoint step sizes without risk to the physical hardware, and confirmed that MoveIt 2's planning pipeline could generate feasible incremental motions at the required update rate.

VII.II.III Hardware Deployment and Servoing Validation

Following successful simulation trials, the visual servoing loop was deployed on the physical robot. An AprilTag was placed at a known location in the workspace, and the system was commanded to servo toward it. We observed the arm's real-world trajectory and compared the end-effector position reported by joint encoders against the tag pose reported by the camera. This stage revealed discrepancies introduced by mechanical backlash, communication latency between the Jetson Orin Nano and the motor controllers, and minor inaccuracies in the URDF kinematic model that were not apparent in simulation. Iterative tuning was performed to adjust servo gains and error thresholds to account for these real-world factors.

VII.II.IV Grasp Pose Parameterization

With the servoing loop functioning reliably on hardware, we turned to tuning the desired grasp pose offset. This offset defines a fixed displacement $\Delta\mathbf{T}_{grasp}$ such that the servoing target becomes $T_{goal} = {}^{c}T_{tag} \cdot \Delta\mathbf{T}_{grasp}$, encoding the desired approach distance, orientation, and

clearance above the object. The parameters of this offset were tuned empirically through repeated real-world grasp trials. We adjusted the offset incrementally, testing each configuration by commanding the robot to servo to the computed grasp pose, close the gripper, and lift. Success was evaluated by whether the object was securely grasped without collision or slippage, and the final parameters were selected based on consistent grasp reliability across multiple trials.



Figure 22: Grasp Pose Estimating from Collecting Camera Localization Data

VII.III Full System Integration Testing

The final validation stage integrated the finite state machine and eye-tracking interface with the manipulation pipeline. A user was seated within the camera's field of view, and the blink detection and gaze estimation modules were activated alongside the AprilTag localizer and servoing controller. We tested the complete interaction loop: the user fixated on a tagged object, issued a triple-blink command, and the system transitioned from observation to retrieval, executed the grasp, and delivered the object. A second triple-blink triggered the return sequence. Testing at this stage focused on the robustness of the state transitions under realistic conditions: including partial occlusions, varied blink speeds across different users, and edge cases such as blinking while not looking at any valid target. The system was iterated upon until state transitions were reliable and the end-to-end pipeline executed consistently without manual intervention.



Figure 23: Full System Implementation Testing with Two Books

VIII Cost Analysis

The total cost of the RoArm-M3-Pro vision-guided manipulation system came to $857.97, covering all hardware necessary for a fully functional autonomous grasping platform. It is worth noting that the Jetson Orin Nano was a pre-purchased item contributed by one of the team members, meaning the effective out-of-pocket expenditure for this project was reduced to approximately $607.98, under the budget for the Project.

We anticipate that the cost of this system could be reduced substantially in a mass production context. The custom power board, which was fabricated as a one-off prototype with individually sourced components, would benefit significantly from economies of scale in both PCB manufacturing and bulk component procurement. Similarly, volume purchasing agreements for the robotic arm and compute hardware would drive per-unit prices down considerably. As edge AI platforms like the Jetson family continue to mature and more affordable alternatives enter the market, the compute cost is expected to decline further. Taken together, these factors suggest that a production variant of this system could be made considerably more accessible, lowering the barrier to entry for researchers, educators, and small-scale automation applications that stand to benefit from low-cost vision-guided manipulation.

Item	**Total Cost**
RoArm-M3 Pro Robotic Arm	$399.99
Jetson Orin Nano Super	$249.99
Cameras, Microphone	$67.96
Power Board PCB, Components	$115.48
12V 9Ah SLA Battery	$24.55
	$857.97

IX Conclusion

The Helping Hand project successfully demonstrated a functional assistive robotic platform that bridges the gap between user intent and physical action for individuals with upper limb impairments. By integrating a custom eye-tracking module with blink detection, an AprilTag-based visual servoing pipeline built on ROS 2 and MoveIt 2, and a voice command interface, the system enables hands-free object retrieval through intuitive, non-invasive input methods. The custom three-stage power conversion board and portable battery solution further support the system's goal of wheelchair-mountable independence. Full system integration testing confirmed that the end-to-end pipeline — from gaze fixation and triple-blink command through robotic grasping and object delivery — operates without manual intervention, and the total hardware cost of approximately $858 positions the platform well below the price range of commercial prosthetics and existing assistive robotics solutions.

Despite these achievements, testing revealed areas for improvement that would strengthen the system's viability for real-world deployment. Pupil detection accuracy was limited by the use of conventional visible-light cameras, which are sensitive to ambient lighting variations; upgrading to infrared sensors, as used in industry-standard eye trackers, would significantly improve gaze estimation precision and consistency. Additionally, expanding the calibration dataset and refining the polynomial gaze-mapping model would reduce the drift observed during extended use. Future work could also explore replacing AprilTag fiducials with general-purpose object detection models to eliminate the need for pre-tagged items, broadening the range of objects the system can retrieve. With these refinements and the cost reductions expected from scaled production, Helping Hand has strong potential to evolve into a practical, affordable tool that restores meaningful autonomy to those who need it most.

X Compliance

The Helping Hand system was designed with adherence to relevant IEEE standards and ethical guidelines throughout its development. The power conversion and delivery subsystem was developed in accordance with IEEE 1725, which governs rechargeable battery safety for portable computing devices, ensuring that the custom charging board incorporates appropriate overcurrent, overvoltage, and thermal protections for the 12V SLA battery. The wireless communication between the ESP32 robot controller and the Jetson Orin Nano complies with IEEE 802.11 b/g/n standards for local wireless networking, ensuring reliable and interoperable data exchange within the system. Additionally, the project followed the IEEE Code of Ethics by prioritizing user safety, accessibility, and dignity in every design decision — from selecting non-invasive input methods over surgical alternatives to engineering fail-safe behaviors within the finite state machine that prevent unintended robotic motion. All software was built on open-source frameworks including ROS 2, MoveIt 2, and OpenCV, consistent with IEEE's support for transparent and reproducible engineering practices.

The team recognizes that deploying an assistive robotic system in close physical proximity to vulnerable users carries significant responsibility. Careful attention was given to ensuring that the robotic arm operates within defined workspace boundaries, that the gripper exerts limited force to prevent injury, and that the triple-blink activation threshold minimizes the risk of accidental commands from natural blinking. While the current prototype serves as a proof of concept validated in a controlled laboratory environment, any future deployment in clinical or home settings would require compliance with additional regulatory frameworks, including IEC 62304 for medical device software lifecycle processes and ISO 13482 for personal care robot safety. The team is committed to pursuing these certifications and conducting formal usability studies with individuals from the target population to ensure that Helping Hand meets the highest standards of safety, reliability, and user-centered design before reaching the people it was built to serve.

XI References

[1] W.H.O., “World Report on Disability,” World Health Organization, https://www.who.int/publications/i/item/9789241564182 (accessed Jun. 22, 2025).

[2] United States CDC, “Disability impacts all of us infographic,” Centers for Disease Control and Prevention, https://www.cdc.gov/disability-and-health/articles-documents/disability-impacts-all-of-us-infographic.html?CDC_AAref_Val=https%3A%2F%2Fwww.cdc.gov%2Fncbddd%2Fdisabilityandhealth%2Finfographic-disability-impacts-all.html (accessed Jun. 22, 2025).

[3] K. D. Anderson, “Targeting recovery: Priorities of the spinal cord-injured population,” Journal of Neurotrauma, https://pubmed.ncbi.nlm.nih.gov/15672628/ (accessed Jun. 23, 2025).

[4] “5 + 1 DOF High-Torque Serial Bus Servo Robotic Arm Kit, Based On ESP32, 2-DOF Wrist Joint, Wireless Control, Compatible With LeRobot,” Waveshare, https://www.waveshare.com/roarm-m3.htm (accessed Jun. 22, 2025).